

# Remove redundant `reserve_hint` members from view classes

- - Abstract
  - Revision history
  - Discussion
  - Proposed change
  - References

## Abstract

This paper removes the `reserve_hint` members of view classes when their `size` members are already provided.

## Revision history

### R1

Only provide the approximately `reserve_hint` for `take_view` when the underlying range is not `approximately_sized_range`.

### R0

Initial revision.

## Discussion

After [P2846](#), `sized_range` always implies `approximately_sized_range` by its definition ([\[range.sized\]](#)):

```
template<class T>
concept sized_range =
    approximately_sized_range<T> && requires(T& t) { ranges::size(t); };
```

And `size` and `reserve_hint` members of the view class are currently with the following constraint:

```
template<range R>
struct some_view : public view_interface<some_view<R>> {
    [...]
    constexpr auto size()          requires sized_range<R>
    constexpr auto size() const    requires sized_range<const R>
    constexpr auto reserve_hint()  requires approximately_sized_range<R>
    constexpr auto reserve_hint() const requires approximately_sized_range<const R>
    [...]
};
```

This turns out that all view classes with a valid `size` member also have a `reserve_hint` member.

However, those `reserve_hint` in such cases is actually redundant because `ranges::reserve_hint` always prefers the `size` member when available ([\[range.prim.size.hint\]](#)):

-2- Given a subexpression `E` with type `T`, let `t` be an lvalue that denotes the reified object for `E`. Then:

(2.1) — If `ranges::size(E)` is a valid expression, `ranges::reserve_hint(E)` is expression-equivalent to `ranges::size(E)`.

(2.2) — Otherwise, if `auto(t.reserve_hint())` is a valid expression of integer-like type (`[iterator.concept.winc]`), `ranges::reserve_hint(E)` is expression-equivalent to `auto(t.reserve_hint())`.

[...]

When an exact size is available, providing an additional, less precise `reserve_hint` serves no practical purpose. There is no use case in the standard that requires these redundant `reserve_hint` members. Both `ranges::to` and standard containers reserve capacity by calling `ranges::reserve_hint`, which already retrieves the best available size estimate.

Additionally, the motivating case in [P2846](#) (`uppercase_view`) provides only a `reserve_hint` member, as it cannot offer an exact size:

```
template<input_range V>
class uppercase_view {
    constexpr const V & base() const;
```

```
constexpr auto begin() const;
constexpr auto end() const;

constexpr auto reserve_hint() requires approximately_sized_range<V> {
    return ranges::reserve_hint(base());
}
constexpr auto reserve_hint() const requires approximately_sized_range<const V> {
    return ranges::reserve_hint(base());
}
};
```

This indicates that `reserve_hint` should only be used to resolve situations where `size` cannot be provided, as it is only reasonable to provide `reserve_hint` if `approximately_sized_range` is modeled and `sized_range` is not; otherwise these `reserve_hint` would lead to unnecessary instantiation overhead. We should not pay for what we do not need.

## Proposed change

This wording is relative to the [N5014](#). It replaces `approximately_sized_range` with the newly introduced exposition-only concept `only-approximately-sized` in all relevant view classes, to remove `reserve_hint` members from views that already provide a `size` member.

1. Modify 25.7.5 [\[range.adaptor.helpers\]](#), as indicated:

```
namespace std::ranges {
    [...]
    template<bool Const, class... Views>
        concept all-forward = // exposition only
            (forward_range<maybe-const<Const, Views>> && ...);

    template<class R>
    concept only-approximately-sized = // exposition only
    !sized_range<R> && approximately_sized_range<R>;
}
```

2. Modify 25.7.6.2 [\[range.ref.view\]](#), as indicated:

```
namespace std::ranges {
    template<range R>
        requires is_object_v<R>
        class ref_view : public view_interface<ref_view<R>> {
            [...]
        public:
            [...]
            constexpr auto reserve_hint() const requires approximately_sized_rangeonly-approximately-sized<R>
            { return ranges::reserve_hint(*r_); }
            [...]
        };
    [...]
}
```

3. Modify 25.7.6.3 [\[range.owning.view\]](#), as indicated:

```
namespace std::ranges {
    template<range R>
        requires movable<R> && (!is_initializer_list<R>) // see [range.refinements]
        class owning_view : public view_interface<owning_view<R>> {
            [...]
        public:
            [...]
            constexpr auto reserve_hint() requires approximately_sized_rangeonly-approximately-sized<R>
            { return ranges::reserve_hint(r_); }
            constexpr auto reserve_hint() const requires approximately_sized_rangeonly-approximately-sized<const R>
            { return ranges::reserve_hint(r_); }
            [...]
        };
}
```

4. Modify 25.7.7.2 [\[range.as.rvalue.view\]](#), as indicated:

```
namespace std::ranges {
    template<view V>
        requires input_range<V>
        class as_rvalue_view : public view_interface<as_rvalue_view<V>> {
            [...]
        public:
            [...]
            constexpr auto reserve_hint() requires approximately_sized_rangeonly-approximately-sized<V>
            { return ranges::reserve_hint(base_); }
            constexpr auto reserve_hint() const requires approximately_sized_rangeonly-approximately-sized<const V>
            { return ranges::reserve_hint(base_); }
        };
    [...]
```

```
}
```

5. Modify 25.7.9.2 [\[range.transform.view\]](#), as indicated:

```
namespace std::ranges {
    template<input_range V, move_constructible F>
        requires view<V> && is_object_v<F> &&
            regular_invocable<F&, range_reference_t<V>> &&
            can_reference_invoke_result_t<F&, range_reference_t<V>>>
    class transform_view : public view_interface<transform_view<V, F>> {
    [...]
    public:
    [...]
        constexpr auto reserve_hint() requires approximately_sized_rangeonly-approximately-sized<V>
        { return ranges::reserve_hint(base_); }
        constexpr auto reserve_hint() const requires approximately_sized_rangeonly-approximately-sized<const V>
        { return ranges::reserve_hint(base_); }
    };
    [...]
}
```

6. Modify 25.7.10.2 [\[range.take.view\]](#), as indicated:

```
namespace std::ranges {
    template<view V>
    class take_view : public view_interface<take_view<V>> {
    [...]
    public:
    [...]
        constexpr auto reserve_hint() noexcept requires (!approximately_sized_range<V>) {
            if constexpr (approximately_sized_range<V>) {
                auto n = static_cast<range_difference_t<V>>(ranges::reserve_hint(base_));
                return to_unsigned_like(ranges::min(n, count_));
            }
            return to_unsigned_like(count_);
        }

        constexpr auto reserve_hint() requires only-approximately-sized<V> {
            auto n = static_cast<range_difference_t<V>>(ranges::reserve_hint(base_));
            return to_unsigned_like(ranges::min(n, count_));
        }

        constexpr auto reserve_hint() const noexcept requires (!approximately_sized_range<const V>) {
            if constexpr (approximately_sized_range<const V>) {
                auto n = static_cast<range_difference_t<const V>>(ranges::reserve_hint(base_));
                return to_unsigned_like(ranges::min(n, count_));
            }
            return to_unsigned_like(count_);
        }

        constexpr auto reserve_hint() const requires only-approximately-sized<const V> {
            auto n = static_cast<range_difference_t<const V>>(ranges::reserve_hint(base_));
            return to_unsigned_like(ranges::min(n, count_));
        }
    };
    [...]
}
```

7. Modify 25.7.12.2 [\[range.drop.view\]](#), as indicated:

```
namespace std::ranges {
    template<view V>
    class drop_view : public view_interface<drop_view<V>> {
    public:
    [...]
        constexpr auto reserve_hint() requires approximately_sized_rangeonly-approximately-sized<V> {
            const auto s = static_cast<range_difference_t<V>>(ranges::reserve_hint(base_));
            return to_unsigned_like(s < count_ ? 0 : s - count_);
        }

        constexpr auto reserve_hint() const requires approximately_sized_rangeonly-approximately-sized<const V> {
            const auto s = static_cast<range_difference_t<const V>>(ranges::reserve_hint(base_));
            return to_unsigned_like(s < count_ ? 0 : s - count_);
        }
    [...]
    };
    [...]
}
```

8. Modify 25.7.20.2 [\[range.common.view\]](#), as indicated:

```
namespace std::ranges {
    template<view V>
```

```

    requires (!common_range<V> && copyable<iterator_t<V>>())
class common_view : public view_interface<common_view<V>> {
    [...]
public:
    [...]
    constexpr auto reserve_hint() requires approximately_sized_rangeonly-approximately-sized<V> {
        return ranges::reserve_hint(base_);
    }
    constexpr auto reserve_hint() const requires approximately_sized_rangeonly-approximately-sized<const V> {
        return ranges::reserve_hint(base_);
    }
};
    [...]
}

```

9. Modify 25.7.21.2 [\[range.reverse.view\]](#), as indicated:

```

namespace std::ranges {
    template<view V>
        requires bidirectional_range<V>
    class reverse_view : public view_interface<reverse_view<V>> {
        [...]
    public:
        [...]
        constexpr auto reserve_hint() requires approximately_sized_rangeonly-approximately-sized<V> {
            return ranges::reserve_hint(base_);
        }
        constexpr auto reserve_hint() const requires approximately_sized_rangeonly-approximately-sized<const V> {
            return ranges::reserve_hint(base_);
        }
    };
    [...]
}

```

10. Modify 25.7.22.2 [\[range.as.const.view\]](#), as indicated:

```

namespace std::ranges {
    template<view V>
        requires input_range<V>
    class as_const_view : public view_interface<as_const_view<V>> {
        [...]
    public:
        [...]
        constexpr auto reserve_hint() requires approximately_sized_rangeonly-approximately-sized<V> {
            return ranges::reserve_hint(base_);
        }
        constexpr auto reserve_hint() const requires approximately_sized_rangeonly-approximately-sized<const V> {
            return ranges::reserve_hint(base_);
        }
    };
    [...]
}

```

11. Modify 25.7.23.2 [\[range.elements.view\]](#), as indicated:

```

namespace std::ranges {
    [...]
    template<input_range V, size_t N>
        requires view<V> && has_tuple_element<range_value_t<V>, N> &&
            has_tuple_element<remove_reference_t<range_reference_t<V>>, N> &&
            returnable_element<range_reference_t<V>, N>
    class elements_view : public view_interface<elements_view<V, N>> {
    public:
        [...]
        constexpr auto reserve_hint() requires approximately_sized_rangeonly-approximately-sized<V>
        { return ranges::reserve_hint(base_); }

        constexpr auto reserve_hint() const requires approximately_sized_rangeonly-approximately-sized<const V>
        { return ranges::reserve_hint(base_); }
        [...]
    };
}

```

12. Modify 25.7.24.2 [\[range.enumerate.view\]](#), as indicated:

```

namespace std::ranges {
    template<view V>
        requires range_with_movable_references<V>
    class enumerate_view : public view_interface<enumerate_view<V>> {
        [...]
    public:
        [...]
        constexpr auto reserve_hint() requires approximately_sized_rangeonly-approximately-sized<V>

```

```

    { return ranges::reserve_hint(base_); }
    constexpr auto reserve_hint() const requires approximately_sized_rangeonly-approximately-sized<const V>
    { return ranges::reserve_hint(base_); }
    [...]
};
[...]
```

13. Modify 25.7.27.2 [\[range.adjacent.view\]](#), as indicated:

```

namespace std::ranges {
    template<forward_range V, size_t N>
        requires view<V> && (N > 0)
    class adjacent_view : public view_interface<adjacent_view<V, N>> {
        [...]
    public:
        [...]
        constexpr auto reserve_hint() requires approximately_sized_rangeonly-approximately-sized<V>;
        constexpr auto reserve_hint() const requires approximately_sized_rangeonly-approximately-sized<const V>;
    };
}

[...]
```

constexpr auto reserve\_hint() requires ~~approximately\_sized\_range~~only-approximately-sized<V>;  
 constexpr auto reserve\_hint() const requires ~~approximately\_sized\_range~~only-approximately-sized<const V>;

-3- *Effects*: Equivalent to:

[...]

14. Modify 25.7.28.2 [\[range.adjacent.transform.view\]](#), as indicated:

```

namespace std::ranges {
    template<forward_range V, move_constructible F, size_t N>
        requires view<V> && (N > 0) && is_object_v<F> &&
            regular_invocable<F&, REPEAT(range_reference_t<V>, N)...> &&
            can_reference_invoke_result_t<F&, REPEAT(range_reference_t<V>, N)...>
    class adjacent_transform_view : public view_interface<adjacent_transform_view<V, F, N>> {
        [...]
    public:
        [...]
        constexpr auto reserve_hint() requires approximately_sized_rangeonly-approximately-sized<InnerView> {
            return inner_.reserve_hint();
        }

        constexpr auto reserve_hint() const requires approximately_sized_rangeonly-approximately-sized<const InnerView> {
            return inner_.reserve_hint();
        }
    };
}

[...]
```

15. Modify 25.7.29.2 [\[range.chunk.view.input\]](#), as indicated:

```

namespace std::ranges {
    [...]
    template<view V>
        requires input_range<V>
    class chunk_view : public view_interface<chunk_view<V, N>> {
        [...]
    public:
        [...]
        constexpr auto reserve_hint() requires approximately_sized_rangeonly-approximately-sized<V>;
        constexpr auto reserve_hint() const requires approximately_sized_rangeonly-approximately-sized<const V>;
    };
    [...]
}

[...]
```

constexpr auto reserve\_hint() requires ~~approximately\_sized\_range~~only-approximately-sized<V>;  
 constexpr auto reserve\_hint() const requires ~~approximately\_sized\_range~~only-approximately-sized<const V>;

-6- *Effects*: Equivalent to:

[...]

16. Modify 25.7.29.4 [\[range.chunk.outer.value\]](#), as indicated:

```

namespace std::ranges {
    [...]
    template<view V>
        requires input_range<V>
    struct chunk_view<V>::outer_iterator::value_type : view_interface<value_type> {
        [...]
    };
}

[...]
```

```

public:
    [...]
    constexpr auto size() const
        requires sized_sentinel_for<sentinel_t<V>, iterator_t<V>>;
    constexpr auto reserve_hint() const noexcept
        requires (!sized_sentinel_for<sentinel_t<V>, iterator_t<V>>);
};
}

[...]

constexpr auto reserve_hint() const noexcept
    requires (!sized_sentinel_for<sentinel_t<V>, iterator_t<V>>);

-5- Effects: Equivalent to:

[...]

```

17. Modify 25.7.29.6 [\[range.chunk.view.fwd\]](#), as indicated:

```

namespace std::ranges {
    template<view V>
        requires forward_range<V>
    class chunk_view : public view_interface<chunk_view<V, N>> {
    public:
        [...]
        constexpr auto reserve_hint() requires approximately_sized_rangeonly-approximately-sized<V>;
        constexpr auto reserve_hint() const requires approximately_sized_rangeonly-approximately-sized<const V>;
    };
}

[...]

constexpr auto reserve_hint() requires approximately_sized_rangeonly-approximately-sized<V>;
constexpr auto reserve_hint() const requires approximately_sized_rangeonly-approximately-sized<const V>;

-4- Effects: Equivalent to:

[...]

```

18. Modify 25.7.30.2 [\[range.slide.view\]](#), as indicated:

```

namespace std::ranges {
    [...]
    template<forward_range V>
        requires view<V>
    class slide_view : public view_interface<slide_view<V, N>> {
    public:
        [...]
        constexpr auto reserve_hint() requires approximately_sized_rangeonly-approximately-sized<V>;
        constexpr auto reserve_hint() const requires approximately_sized_rangeonly-approximately-sized<const V>;
    };
}

[...]

constexpr auto reserve_hint() requires approximately_sized_rangeonly-approximately-sized<V>;
constexpr auto reserve_hint() const requires approximately_sized_rangeonly-approximately-sized<const V>;

-10- Effects: Equivalent to:

[...]

```

19. Modify 25.7.32.2 [\[range.stride.view\]](#), as indicated:

```

namespace std::ranges {
    [...]
    template<input_range V>
        requires view<V>
    class stride_view : public view_interface<stride_view<V, N>> {
    public:
        [...]
        constexpr auto reserve_hint() requires approximately_sized_rangeonly-approximately-sized<V>;
        constexpr auto reserve_hint() const requires approximately_sized_rangeonly-approximately-sized<const V>;
    };
}

[...]

constexpr auto reserve_hint() requires approximately_sized_rangeonly-approximately-sized<V>;
constexpr auto reserve_hint() const requires approximately_sized_rangeonly-approximately-sized<const V>;

```

-5- *Effects*: Equivalent to:

[...]

20. Modify 25.7.34.2 [\[range.cache.latest.view\]](#), as indicated:

```
namespace std::ranges {
    template<input_range V>
        requires view<V>
        class cache_latest_view : public view_interface<cache_latest_view<V, N>> {
            [...]
        public:
            [...]
            constexpr auto reserve_hint() requires approximately_sized_rangeonly-approximately-sized<V>;
            constexpr auto reserve_hint() const requires approximately_sized_rangeonly-approximately-sized<const V>;
        };
        [...]
    }
```

[...]

```
constexpr auto reserve_hint() requires approximately_sized_rangeonly-approximately-sized<V>;
constexpr auto reserve_hint() const requires approximately_sized_rangeonly-approximately-sized<const V>;
```

-5- *Effects*: Equivalent to: `return ranges::reserve_hint(base_);`

[...]

21. Modify 25.7.35.2 [\[range.to.input.view\]](#), as indicated:

```
namespace std::ranges {
    template<input_range V>
        requires view<V>
        class to_input_view : public view_interface<to_input_view<V, N>> {
            [...]
        public:
            [...]
            constexpr auto reserve_hint() requires approximately_sized_rangeonly-approximately-sized<V>;
            constexpr auto reserve_hint() const requires approximately_sized_rangeonly-approximately-sized<const V>;
        };
        [...]
    }
```

[...]

```
constexpr auto reserve_hint() requires approximately_sized_rangeonly-approximately-sized<V>;
constexpr auto reserve_hint() const requires approximately_sized_rangeonly-approximately-sized<const V>;
```

-6- *Effects*: Equivalent to: `return ranges::reserve_hint(base_);`

## References

[P2846R6]

Corentin Jabot. `reserve_hint`: Eagerly reserving memory for not-quite-sized lazy ranges. URL: <https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2025/p2846r6.pdf>